

Laboratorio 3 — Algoritmos de Enrutamiento

Dijkstra, Flooding y Link State Routing

Leonardo Dufrey Mejía Mejía

María José Girón Isidro

Ernesto David Ascencio

Hugo Daniel Barillas

Universidad del Valle de Guatemala — CC3067 Redes

Introducción

Algoritmo de Dijkstra

- Propósito

- Funcionamiento

- Modelo de topología

- Modo standalone

- Pruebas

Algoritmo Flooding

- Propósito

- Descubrimiento de vecinos

- Reenvío por inundación

- Modularidad e integración

- Modo standalone

- Pruebas

Infraestructura de red

- Propósito

- Transporte

- Forwarding

- Health check

- Modo en vivo

- Pruebas

Link State Routing

- Propósito

- Formato del LSP

- Distribución por flooding

- Topología derivada y tabla de ruteo

- Actualización ante cambios

- Modo standalone

- Pruebas

Introducción

Este documento describe la implementación de tres algoritmos de enrutamiento sobre una misma infraestructura de red: **Dijkstra** (ruteo con topología conocida), **Flooding** (inundación sin tabla de rutas) y **Link State Routing** (LSR), que combina a los dos anteriores. Cada nodo se ejecuta como un proceso independiente que se comunica por sockets TCP usando un protocolo JSON común.

El proyecto está dividido en módulos con un solo dueño cada uno. Los algoritmos son puros —reciben y devuelven estructuras de datos, nunca abren sockets— para poder probarlos de forma aislada y para que LSR pueda reutilizar Dijkstra y Flooding sin modificarlos. La capa de red (sockets, hilos, forwarding, health check) conoce únicamente el contrato `RoutingAlgorithm` de `shared/interfaces.py`.

Las secciones siguientes describen cada algoritmo y la infraestructura que los conecta. Todas las pruebas se ejecutan con `python -m pytest tests/ -q` desde la raíz del repositorio.

Algoritmo de Dijkstra

Propósito

Dijkstra calcula el camino de menor costo desde un nodo de origen hacia los demás nodos de una red. El cálculo usa una topología completa con nodos, enlaces y el peso de cada enlace. En este laboratorio, el peso representa el costo de atravesar una conexión.

El algoritmo requiere pesos no negativos. La implementación rechaza un enlace con peso negativo para evitar resultados incorrectos.

Funcionamiento

El cálculo comienza con costo cero para el origen y costo infinito para los demás nodos. Una cola de prioridad selecciona el nodo pendiente con menor costo conocido. Después se revisan sus vecinos y se actualiza una ruta cuando el nuevo costo es menor que el anterior.

Cada actualización conserva el primer salto desde el origen. Ese dato permite reenviar un paquete sin guardar la ruta completa. El proceso termina cuando la cola queda vacía.

Para cada destino, `shortest_paths` devuelve el costo total y el siguiente salto. Los destinos aislados conservan costo infinito y no tienen siguiente salto. `build_routing_table` elimina esos destinos y produce el formato que consumirá el módulo de forwarding:

```
{  
    "B": "C",
```

```

    "C": "C",
}

```

En este ejemplo, un paquete dirigido a B debe salir primero por C. Un paquete dirigido a C se entrega directamente a ese vecino.

Con una cola de prioridad, el tiempo de ejecución es $O((V + E) \log V)$, donde V es la cantidad de nodos y E es la cantidad de enlaces. La memoria utilizada es $O(V + E)$.

Modelo de topología

La clase `Topology` guarda una lista de adyacencia no dirigida. `add_edge` registra el enlace en ambos sentidos y `get_neighbors` devuelve los vecinos con sus pesos.

`DijkstraRoutingAlgorithm` guarda la lista de aristas base y un conjunto de nodos caídos. Ante `handle_neighbor_down` agrega el nodo al conjunto y ante `handle_neighbor_up` lo quita; en ambos casos reconstruye la topología desde las aristas base omitiendo los caídos y vuelve a correr Dijkstra. Al no mutar la topología de forma destructiva, un vecino que se recupera reaparece con todos sus enlaces.

`Topology.from_json` acepta una sección con este formato:

```

{
  "nodes": ["A", "B", "C"],
  "edges": [
    {"node_a": "A", "node_b": "B", "weight": 4},
    {"node_a": "A", "node_b": "C", "weight": 1},
    {"node_a": "C", "node_b": "B", "weight": 2}
  ]
}

```

El método también puede recibir el archivo completo de configuración. En ese caso, toma los datos incluidos bajo la clave `topology`. Esta decisión permite usar el mismo modelo en el modo estático y, más adelante, con la topología construida por Link State Routing.

Modo standalone

El punto de entrada carga el nodo de origen y la topología desde el archivo JSON. Luego calcula e imprime el costo y el siguiente salto de cada destino:

```

python -m node.main --config shared/config/topology_example.json
                    --mode dijkstra

```

La configuración incluida produce esta salida:

```

Rutas desde A
destino costo  siguiente salto
B           7           B

```

C	7	C
I	1	I

Este modo no abre sockets. Su función durante la Fase 1 es comprobar el cálculo estático antes de conectarlo con la infraestructura de red.

Pruebas

Las pruebas cubren la carga de una topología, una ruta indirecta más barata, un nodo inalcanzable, el recálculo después de una caída y la ejecución del modo standalone.

```
python -m pytest tests/test_dijkstra.py -q
```

Algoritmo Flooding

Propósito

Flooding distribuye un paquete sin necesitar la topología completa ni una tabla de rutas. Cada nodo conoce únicamente a sus vecinos directos y reenvía una copia a todos los vecinos activos, excepto al que entregó el paquete. Esta característica permite utilizarlo como algoritmo standalone y como mecanismo de distribución de paquetes LSP en Link State Routing.

El costo de esta simplicidad es la generación de copias. Una red con ciclos podría reenviar el mismo paquete indefinidamente, por lo que la implementación combina un límite de saltos (TTL) con detección de duplicados.

Descubrimiento de vecinos

La clase `NeighborTable`, ubicada en `neighbor_discovery.py`, recibe la lista de vecinos configurados del nodo. Cada entrada guarda el identificador, IP, puerto, peso, estado, último instante de actividad y retardo observado. Los vecinos empiezan inactivos hasta confirmar su presencia mediante un paquete hello.

Los paquetes de descubrimiento utilizan el protocolo JSON compartido:

```
{
  "proto": "flooding",
  "type": "hello",
  "from": "A",
  "to": "B",
  "ttl": 1,
  "headers": [],
  "payload": {
    "node_id": "A",
    "ip": "127.0.0.1",
    "port": 5000,
```

```
    "sent_at": 10.5
  }
}
```

Al recibir el paquete, `on_hello_received` marca activo al emisor, actualiza su dirección y calcula un retardo aproximado como la diferencia entre recepción y envío. `expire_stale` marca como caído un vecino que no ha enviado actividad dentro del timeout. La tabla utiliza un bloqueo reentrante para permitir su consulta desde los hilos de forwarding y health check.

Reenvío por inundación

El procesamiento sigue cuatro pasos:

1. Obtener el identificador del paquete desde `headers.packet_id`. Si no existe, se calcula una huella SHA-256 estable con los campos del paquete, excluyendo el TTL.
2. Rechazar el paquete si ya fue procesado o si su TTL es menor o igual que 1.
3. Seleccionar todos los vecinos activos excepto el vecino que entregó el paquete.
4. Crear una copia por destino con el TTL decrementado, sin modificar el paquete original.

Excluir el TTL de la huella es importante porque su valor cambia en cada salto. Para mensajes distintos que tengan exactamente el mismo contenido se recomienda asignar un `packet_id` único en los encabezados.

Si un nodo tiene grado d , seleccionar destinos y crear copias requiere $O(d)$ tiempo. Consultar o registrar un identificador visto tiene tiempo esperado $O(1)$. La memoria de duplicados crece $O(p)$, donde p es el número de paquetes procesados durante la ejecución.

Modularidad e integración

`FloodingRoutingAlgorithm` implementa el contrato común `RoutingAlgorithm`. El método `flood` devuelve pares (vecino, paquete) listos para envío, pero nunca abre sockets. De esta forma, `forwarding.py` puede transmitirlos durante la Fase 3 y el módulo LSR puede reutilizar la misma lógica sin modificarla.

La clase también expone los eventos `handle_neighbor_up` y `handle_neighbor_down`, una cola de paquetes salientes y consulta directa de vecinos activos. El estado compartido se protege con bloqueos para soportar la ejecución asíncrona requerida por la práctica.

Modo standalone

El modo standalone carga únicamente el nodo y sus vecinos desde la configuración, inicializa el algoritmo y prepara un paquete HELLO para cada vecino:

```
python -m node.main --config shared/config/topology_example.json
    --mode flooding
```

Este modo valida el arranque, la lectura de vecinos y la generación de descubrimiento sin transmitir por la red. Con `--live`, los paquetes pendientes se entregan al proceso de forwarding y se envían por sockets reales; un mensaje de usuario se inunda salto a salto hasta su destino. Se probó con tres nodos A–B–C: un mensaje de A a C (sin enlace directo) llega inundado a través de B.

Pruebas

Las pruebas unitarias verifican el control de TTL, el rechazo de duplicados, la exclusión del emisor, la inmutabilidad del paquete original, la construcción y recepción de HELLO, la medición de retardo, la caída por timeout, la interfaz del algoritmo y la ejecución standalone.

```
python -m pytest tests/test_flooding.py -q
```

Infraestructura de red

Propósito

Dijkstra y Flooding calculan rutas o deciden reenvíos, pero ninguno abre un socket. La Fase 3 es la capa que los conecta con la red real: escuchar paquetes entrantes, enviarlos por IP:puerto, decidir qué hacer con cada uno según su tipo, y detectar cuándo un vecino se cae o se recupera. Todo el código de esta fase conoce únicamente el contrato `RoutingAlgorithm` de `shared/interfaces.py`; nunca importa una clase concreta de Dijkstra, Flooding o LSR.

Transporte

`node/network/socket_manager.py` usa TCP con un patrón de un paquete por conexión: `connect` → enviar el JSON serializado (`shared/protocol.serialize`) → cerrar. No se mantienen conexiones persistentes entre nodos.

- `start_listening(on_packet_received)` abre un socket servidor en un hilo daemon. Cada conexión aceptada se atiende en su propio hilo, que lee hasta que el emisor cierra la conexión y entrega el contenido crudo al callback.
- `send(to_ip, to_port, packet)` abre una conexión con timeout, envía y cierra. Si el vecino no responde (caído, timeout, puerto cerrado), la excepción de socket se traduce a `NeighborUnreachableError`, para que el resto del sistema no dependa de excepciones crudas de socket.
Forwarder._send_to_neighbor la atrapa: un vecino que todavía no arranca o que se acaba de caer no debe tumbar el nodo; el health check ya lo detecta y re-intenta.

Forwarding

`node/network/forwarding.py` define la clase `Forwarder`, que junta al algoritmo activo, la tabla de ruteo compartida, el `SocketManager` y el mapa de direcciones de los vecinos configurados. `handle_incoming_packet` despacha cada paquete según `type`:

- **message**: si el destino es este nodo, se imprime. Si no y el algoritmo expone `flood` (Flooding), se envía una copia por cada vecino activo salvo el emisor. En caso contrario se busca un único siguiente salto en `RoutingTable` (Dijkstra, LSR). En ambos caminos el TTL se decrementa y un TTL agotado o un destino sin ruta conocida se descartan silenciosamente.
- **info**: se delega en `algorithm.handle_info_packet`, que es el único método de la interfaz pensado para que cada algoritmo actualice su estado interno (LSP, vector de distancias, etc.).
- **hello**: si el algoritmo expone `handle_hello_packet` (como Flooding y LSR) se usa; si no, se cae a `algorithm.handle_neighbor_up`, que sí es parte del contrato común.

Después de procesar un `info` o un `hello`, `Forwarder` reenvía lo que el algoritmo haya dejado pendiente en `get_outgoing_packets()` y llama a `sync_routing_table()`.

Por qué `forward_data_packet` no llama al algoritmo directamente

`node/routing/routing_table.py` es la estructura pensada para desacoplar el hilo de routing (que actualiza al algoritmo) del hilo de forwarding (que reenvía datos). `forward_data_packet` solo lee de `RoutingTable`, nunca del algoritmo. `sync_routing_table` es quien traduce el estado del algoritmo a esa tabla, usando exclusivamente `algorithm.get_next_hop(destino)` —el método que la interfaz documenta explícitamente para este propósito— para cada destino conocido (los vecinos configurados, más las claves de `algorithm.routing_table` cuando ese atributo existe, como en Dijkstra y LSR). Así, el módulo de forwarding no necesita conocer los internals de cada algoritmo concreto.

Health check

`node/network/health_check.py` define `HealthChecker`, que cada `interval_seconds` intenta enviar un HELLO a cada vecino configurado. El HELLO lleva en su payload `node_id`, `ip`, `port` y `sent_at`, de modo que el receptor (`NeighborTable.on_hello_received`) puede estimar el retardo del enlace. Un envío que falla (`NeighborUnreachableError`) cuenta como intento fallido; tras `max_failures` fallos consecutivos el vecino se marca caído y se notifica vía `on_status_change`. Si un vecino caído vuelve a responder, se notifica la recuperación. Un callback opcional `on_tick` se ejecuta una vez por ciclo; `node/main.py` lo usa para el re-anuncio periódico del LSP en modo LSR.

`HealthChecker` es agnóstico del algoritmo de ruteo: en `node/main.py`, `on_status_change` es quien llama `algorithm.handle_neighbor_up/handle_neighbor_down` y dispara `forwarder.sync_routing_table()`.

Modo en vivo

```
python -m node.main --config shared/config/topology_example.json
        --mode flooding --live
```

Con `--live`, el nodo arma `RoutingTable`, `SocketManager`, `Forwarder` y `HealthChecker`, empieza a escuchar, envía los paquetes iniciales que el algoritmo tenga pendientes (por ejemplo, los HELLO de Flooding o los LSP de LSR) y arranca el chequeo de salud. Luego entra en un loop interactivo por stdin: una línea destino: mensaje construye y envía un paquete message real. Ctrl+C o EOF detiene el health check y cierra el socket antes de salir.

Sin `--live`, el comportamiento es idéntico al de las fases anteriores: calcula e imprime un resumen estático sin abrir sockets.

Pruebas

Cada módulo tiene su propio archivo de pruebas, con dobles de prueba (`SocketManager/RoutingAlgorithm` falsos) para no depender de la red real salvo en `test_socket_manager.py`, que sí abre sockets TCP reales en `127.0.0.1` con puertos efimeros.

```
python -m pytest tests/test_routing_table.py tests/
        test_socket_manager.py tests/test_forwarding.py tests/
        test_health_check.py -q
```

Link State Routing

Propósito

Link State Routing (LSR) combina los dos algoritmos anteriores sin reimplementar ninguno. Cada nodo describe sus enlaces directos en un *Link State Packet* (LSP), lo distribuye a toda la red con el módulo de Flooding y, cuando ya conoce los LSPs de los demás, reconstruye la topología completa y corre Dijkstra sobre ella para obtener su tabla de ruteo. Si llega un LSP más nuevo —por una caída, un enlace nuevo o un cambio de peso— la topología se reconstruye y las rutas se recalculan.

El módulo vive en `node/algorithms/lsr/` y solo importa código ajeno: `should_forward`, `get_forward_targets` y `decrement_ttl` de Flooding, `NeighborTable` para el estado de vecinos, y `Topology` con `build_routing_table` de Dijkstra.

Formato del LSP

El LSP viaja en el campo `payload` de un paquete con `proto="lsr"` y `type="info"`:


```

{
  "proto": "lsr",
  "type": "info",
  "from": "A",
  "to": "B",
  "ttl": 8,
  "headers": [{"packet_id": "A-3"}],
  "payload": {
    "node_id": "A",
    "sequence": 3,
    "neighbors": [
      {"node_id": "B", "weight": 7},
      {"node_id": "I", "weight": 1}
    ]
  }
}

```

`node_id` es el nodo que originó el LSP y `sequence` distingue las versiones de su información de enlaces. Se siembra con el reloj (`int(time.time())`) al arrancar y crece con cada cambio, de modo que un nodo que reinicia sigue emitiendo LSPs “más nuevos” que los que sus vecinos guardaron. `build_lsp` arma el payload, `parse_lsp` valida un LSP recibido y devuelve una copia normalizada, e `is_newer` compara secuencias para decidir si un LSP reemplaza al que ya se tenía guardado.

El encabezado lleva un `packet_id` con valor “<origen>–<secuencia>” para que los logs y la infraestructura puedan identificar cada LSP.

Distribución por flooding

`LinkStateRouter.broadcast_own_lsp` incrementa la secuencia, arma el LSP con los vecinos activos según `NeighborTable`, lo guarda en la base local (`lsp_db`), recalcula las rutas y encola una copia del paquete para cada vecino activo. Además de anunciarse ante un cambio, el nodo re-anuncia su LSP periódicamente (sobre el mismo temporizador del health check) para recuperar LSPs perdidos en la red.

`handle_info_packet` procesa un LSP entrante en dos pasos:

1. `parse_lsp` y `on_lsp_received`: si el LSP es más nuevo que el guardado, se almacena y se dispara el recálculo; si es viejo o repetido, `on_lsp_received` devuelve `False` y el LSP no se propaga. La deduplicación se apoya en `lsp_db + is_newer` (por (`origen`, `secuencia`)), sin un conjunto de identificadores que crezca sin límite.
2. Reenvío: si el LSP aportó información nueva y su TTL lo permite, se usa `get_forward_targets` —vecinos activos excepto el que entregó el paquete— y cada copia sale con el TTL decrementado (`decrement_ttl` de Flooding), el campo `from` puesto al nodo actual —el último salto— y el `to` al vecino destino. El originador real se conserva dentro de `payload.node_id`.

El módulo de Flooding se usa tal como quedó; LSR solo decide cuándo un LSP amerita reenvío.

Topología derivada y tabla de ruteo

`build_topology_from_lsps` recorre todos los LSPs guardados y agrega cada enlace a una Topology de Dijkstra. Un enlace (X, Y) se incluye si Y no lo contradice: si Y todavía no publicó su LSP se acepta el enlace anunciado por X, y si Y ya publicó pero no lista a X, el enlace se descarta. Así, un nodo caído cuyo LSP viejo sigue en la base queda aislado en cuanto sus vecinos anuncian un LSP sin él.

`recompute_routing_table` corre `build_routing_table` de Dijkstra sobre esa topología y guarda el resultado en `routing_table` (`{destino: siguiente_salto}`), que es el atributo que `forwarding.py` sincroniza hacia la `RoutingTable` compartida mediante `get_next_hop`.

Actualización ante cambios

- **LSP más nuevo:** `on_lsp_received` compara secuencias con `is_newer`, reemplaza el LSP y recalcula. Los LSPs viejos o duplicados no alteran el estado.
- **Vecino caído o recuperado:** `handle_neighbor_down` y `handle_neighbor_up` actualizan `NeighborTable` y llaman a `broadcast_own_lsp`, de modo que el nodo publica un LSP nuevo con la lista de enlaces corregida y la red converge.

Modo standalone

```
python -m node.main --config shared/config/topology_example.json  
                --mode lsr
```

Sin sockets, el nodo solo conoce sus propios enlaces, así que imprime su LSP y la tabla de ruteo derivada de él (los vecinos directos):

```
LSP de A (seq 1788415869)  
B    7  
I    1  
C    7  
Rutas desde A  
destino siguiente salto  
B    B  
C    C  
I    I
```

La secuencia es un timestamp, por eso no arranca en 1. Con `--live` se arma la infraestructura de la Fase 3 (`SocketManager`, `Forwarder`, `HealthChecker`): los LSPs se propagan por la red real, la topología se completa a medida que llegan y la tabla converge al camino de menor costo. Se probó con tres nodos A-B-C: un mensaje de A a C viaja por B, y al detenerse B el health check lo detecta, A re-anuncia su LSP y la ruta se recalcula al enlace directo A-C.

Pruebas

Las pruebas cubren la construcción y lectura del LSP, la comparación por secuencia, la reconstrucción de la topología al llegar un LSP nuevo, el recálculo de la tabla, el descarte de LSPs viejos, el reenvío sin duplicados y excluyendo al emisor, el reanuncio y reruteo cuando cae un vecino, y la ejecución del modo standalone.

```
python -m pytest tests/test_lsr.py -q
```